HEIG-VD HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

Prof. Yann Thoma

Formal verification
Verification of a counter
CERN training - November 2026

Context

We are interested in verifying the correctness of a counter having the following interface :

```
entity counter is
generic (
    SIZE  : integer := 8;
    ERRNO : integer := 0);
port(
    rst_i      : in  std_logic;
    clk_i      : in  std_logic;
    d_i        : in  std_logic_vector(SIZE-1 downto 0);
    en_i       : in  std_logic;
    up_ndown_i : in  std_logic;
    load_i     : in  std_logic;
    value_o    : out std_logic_vector(SIZE-1 downto 0)
);
end counter ;
```

This counter has an enable for incrementing (and decrementing) and the possibility to load a value. Please note that the load has precedence over the enable, and that the `up_ndown_i` selects the operation (increment or decrement).
Propose a set of assertions that allows you to use formal verification to prove the correct behavior of this counter.
The generic parameter `ERRNO` selects the implementation used to test the assertions. `ERRNO = 0` selects the intended counter behavior; values from 1 to 4 each inject a design error. These are verification configurations, not normal operating modes.
The `counter.sby` file defines several tasks : `bmc0`–`bmc4` run bounded checks and `prove0`–`prove4` run proofs for the corresponding `ERRNO` values. The nonzero-error tasks are marked as expected to fail, because they check that the assertions detect the injected faults. Running SBY without a task name runs all configured tasks. To run just one task, pass its name; for example, from the scripts directory use `sby -f counter.sby bmc0` to check the intended implementation, or `sby -f counter.sby prove3` to test one injected error with the proof task.